Pourquoi 0,1 + 0,2 ne vaut-il pas exactement 0,3 en Python ?

1. Une écriture binaire infinie pour 0,1

On part de l'égalité exacte $0{,}1 = 1{,}6 \times 0{,}0625 = 1{,}6 \times 2^{-4}$. En décomposant $1{,}6 = 1 + 0{,}5 + 0{,}1$:

$$
\begin{aligned}
0{,}1 &= 1 \times 2^{-4} + (2^{-1} + 0{,}1) \times 2^{-4} \\
&= 2^{-4} + 2^{-5} + 0{,}1 \times 2^{-4}
\end{aligned}
$$

En remplaçant le 0,1 de droite par la même expression :

$$
\begin{aligned}
0{,}1 &= 2^{-4} + 2^{-5} + (2^{-4} + 2^{-5} + 0{,}1 \times 2^{-4}) \times 2^{-4} \\
&= 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 0{,}1 \times 2^{-8}
\end{aligned}
$$

On recommence indéfiniment : les positions des chiffres 1 se répètent tous les quatre rangs.

$$0{,}1_{10} = 0{,}0001100110011001100110011\ldots_2$$
$$0{,}2_{10} = 0{,}0011001100110011001100110\ldots_2$$

Ces deux écritures sont exactes mais infinies. Un ordinateur ne peut en stocker qu'un nombre fini de chiffres binaires.

2. Les valeurs réellement stockées par Python

En Python, les flottants ordinaires utilisent généralement le format IEEE 754 « binary64 » : 53 bits de précision significative (dont un bit implicite). Les valeurs stockées sont :

```
0,1 stocké = 0x1.999999999999ap-4
0,2 stocké = 0x1.999999999999ap-3
```

La notation hexadécimale ci-dessus donne exactement les valeurs binaires stockées. Elles sont légèrement supérieures à 0,1 et 0,2.

Pour additionner ces deux nombres sans perdre de précision, on peut les exprimer avec un même dénominateur :

$$
\begin{aligned}
0{,}1 \text{ stocké} &= 3602879701896397 / 2^{55} \\
0{,}2 \text{ stocké} &= 7205759403792794 / 2^{55}
\end{aligned}
$$

Leurs numérateurs sont des entiers ; nous pouvons donc effectuer une addition binaire ordinaire.

3. L’addition binaire, bit par bit

Les trois lignes suivantes sont des entiers binaires de 54 bits, tous multipliés par 2^{-55}. Les zéros initiaux servent uniquement à aligner les colonnes.

```
   0011001100110011001100110011001100110011001100110011001101   (0,1)
 + 0110011001100110011001100110011001100110011001100110011010   (0,2)
   ------------------------------------------------------
   1001100110011001100110011001100110011001100110011001100111   (somme exacte)
```

Cette addition exacte des deux valeurs déjà arrondies produit un entier de 54 bits. Or le résultat normalisé doit tenir sur 53 bits significatifs : il faut donc supprimer un bit de poids faible en appliquant la règle d’arrondi IEEE 754.

4. Pourquoi arrondit-on vers le haut ?

```
Somme exacte (54 bits) : 1001100110011001100110011001100110011001100110011001100111
53 bits conservés      :  100110011001100110011001100110011001100110011001100110011
Bit abandonné          :                                                       1
```

Le bit abandonné vaut 1 et aucun autre bit ne suit : on est exactement à mi-chemin entre deux nombres représentables. La règle IEEE 754 par défaut est « au plus proche, et en cas d’égalité, vers celui dont le dernier bit est pair » (round ties to even).

Ici, le dernier bit conservé vaut 1 (impair) : on ajoute donc 1 à la mantisse conservée.

```
Avant arrondi : 1001100110011001100110011001100110011001100110011001100110011
Après arrondi : 1001100110011001100110011001100110011001100110011001100110100
```

La valeur finalement stockée est donc :

```
0x1.3333333333334p-2
= 0.300000000000000044408920985006261616945266723632812 5
```

Python affiche la représentation décimale courte qui permet de retrouver exactement ce flottant :

```
>>> 0.1 + 0.2
0.30000000000000004
```

5. À retenir

La somme mathématique 0,1 + 0,2 vaut exactement 0,3. Mais l’ordinateur additionne des approximations binaires de 0,1 et 0,2, puis arrondit le résultat à 53 bits significatifs. Il obtient un flottant légèrement supérieur à 0,3.

Pour vérifier dans Python : `(0.1 + 0.2).hex()` donne `0x1.3333333333334p-2`, tandis que `(0.3).hex()` donne `0x1.3333333333333p-2`.